

Deadlock Detection, Recovery and Trade-offs

FOSD – Unit 4: Deadlock Modeling and System-level Handling

Prashant Jadav

Department of Computer Engineering
DEPSTAR – CHARUSAT

Learning Objectives

- Explain deadlock detection and why a system may detect rather than prevent or avoid deadlock.
- Construct and analyze a **wait-for graph**.
- Detect deadlock with single and multiple resource instances.
- Compare process termination, resource preemption and rollback.
- Analyze detection frequency, recovery cost and starvation trade-offs.

Deadlock Detection: Basic Idea

Detection approach

The system allows resource allocation to proceed, then checks whether the current state contains a deadlock.

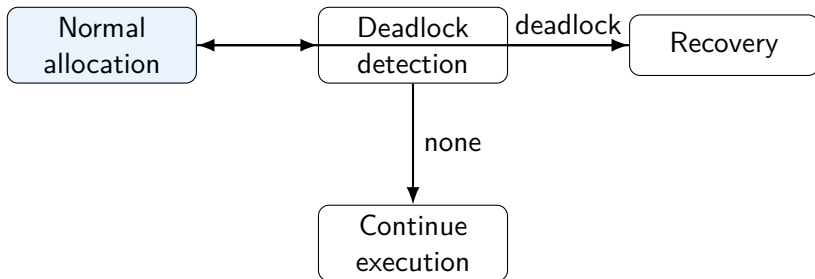
- ① Allow resource allocation.
- ② Run a detection algorithm periodically or when appropriate.
- ③ Identify deadlocked processes/resources.
- ④ Invoke a recovery mechanism.

Allocate → Detect → Recover

Key point

Detection does not prevent the deadlock; it identifies the condition after it occurs and then attempts to break it.

Where Does Detection Fit?



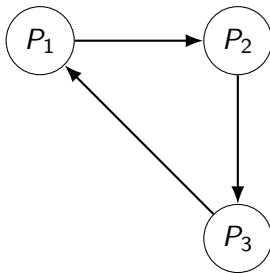
Design question

Frequent detection gives faster discovery but increases overhead. Infrequent detection reduces overhead but can allow a deadlock to persist longer.

Wait-for Graph

A **Wait-for Graph (WFG)** represents waiting relationships directly between processes.

- Each node represents a process.
- An edge $P_i \rightarrow P_j$ means P_i is waiting for a resource held by P_j .
- Resource nodes are omitted.

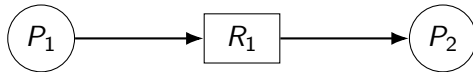


Single-instance case

A cycle in the wait-for graph indicates deadlock.

From Resource Allocation Graph to WFG

Suppose P_1 requests R_1 , while R_1 is allocated to P_2 :



The two RAG edges imply:

$$P_1 \rightarrow P_2$$

Thus the resource node can be eliminated and the waiting relationship represented directly.

Single-Instance Deadlock Detection

Assume every resource type has exactly one instance.

Procedure

- 1 Construct the wait-for graph.
- 2 Search for a directed cycle.
- 3 If a cycle exists, the corresponding processes are deadlocked.

$$P_1 \rightarrow P_2 \rightarrow P_3 \rightarrow P_1$$

Cycle $\Rightarrow$ Deadlock

Standard graph traversal such as DFS can be used for cycle detection.

Single-Instance Detection: Example

No deadlock

$$P_1 \rightarrow P_2 \rightarrow P_3 \rightarrow P_4$$

No edge returns to an earlier process, so there is no cycle.

Deadlock

If $P_4 \rightarrow P_2$, then:

$$P_2 \rightarrow P_3 \rightarrow P_4 \rightarrow P_2$$

is a cycle, so P_2 , P_3 and P_4 are deadlocked.

Multiple Instances: Why a Cycle Is Not Enough

With multiple instances of a resource type, a cycle does not necessarily imply deadlock.

Example

Suppose R has two instances. A process may wait for another instance while one instance is held by a process that can finish and release it.

Therefore:

Cycle $\nRightarrow$ Deadlock necessarily

Detection must consider:

- Available resource instances.
- Current Allocation.
- Current outstanding Request.

Multiple-Instance Detection Algorithm

Let:

$$Work = Available$$

Initially:

$$Finish_i = \begin{cases} false, & Allocation_i \neq 0 \\ true, & Allocation_i = 0 \end{cases}$$

Find P_i such that:

$$Finish_i = false \quad \text{and} \quad Request_i \leq Work.$$

Then assume P_i finishes:

$$Work = Work + Allocation_i, \quad Finish_i = true.$$

Repeat until no further process can be selected.

Multiple-Instance Detection: Decision

No deadlock

If every process can be marked finished, the current state does not contain a deadlock according to the detection test.

Deadlock

If some processes remain with $Finish_i = false$, those processes are deadlocked under the current allocation/request state.

Important: the detection algorithm uses current outstanding requests, not the declared maximum demands used by Banker's Algorithm.

Deadlock Detection vs. Prevention and Avoidance

	Prevention	Avoidance	Detection
Deadlock allowed?	No	No	Yes
Main idea	Break a condition	Stay safe	Detect after occurrence
Maximum demand	Not essential	Typically required	Not required in same form
Runtime cost	Policy restrictions	Allocation safety checks	Detection checks
Recovery needed?	No	No	Yes

Detection accepts risk and pays for recovery

Deadlock Recovery

Once a deadlock is detected, the waiting cycle must be broken.

Three major approaches

- 1 **Process termination**
- 2 **Resource preemption**
- 3 **Rollback**

Detect → Select recovery action → Break cycle → Resume

The appropriate method depends on the resource type, process characteristics and recovery cost.

Recovery 1: Process Termination

Approach

Terminate one or more processes involved in the deadlock so their resources are released.

Two choices:

- ① Abort **all** deadlocked processes.
- ② Abort **one process at a time** until the cycle disappears.

Trade-off

Aborting all is simple but may lose substantial work. Aborting one at a time can reduce loss but requires repeated detection.

Selecting a Process to Terminate

Possible victim-selection factors:

- Process priority.
- Amount of work already completed.
- Estimated remaining execution time.
- Resources currently held.
- Resources still required.
- Number of other processes affected.
- Whether the process is interactive or batch.

Objective

Break the deadlock while minimizing the overall cost of terminating processes.

Recovery 2: Resource Preemption

Approach

Temporarily take a resource from one process and give it to another process so the deadlock cycle is broken.

- ① Select a victim process/resource.
- ② Preempt the resource.
- ③ Restore the victim to a consistent state.
- ④ Give the resource to another process.
- ⑤ Resume the victim later.

Limitation

Not every resource is safely preemptible. A partially completed printer operation, for example, may be difficult to restart consistently.

Victim Selection for Preemption

The system should minimize the total cost of preemption.

- Number of resources held.
- Cost of preempting the selected resource.
- Amount of computation already performed.
- Restart cost.
- Process priority.
- Effect on other processes.

Starvation issue

If the same process is repeatedly selected as the victim, it may never complete.

Therefore, previous preemptions/rollbacks can be included in victim selection.

Recovery 3: Rollback

Definition

Rollback returns a process to an earlier checkpoint or known safe state and restarts execution from that point.

Checkpoint → Execution → Deadlock → Rollback → Restart

- Checkpoints must exist before recovery is needed.
- Work after the selected checkpoint may be lost.
- Resources can be allocated differently during re-execution.

Rollback Example

Suppose a transaction creates checkpoints:

$$S_0 \rightarrow S_1 \rightarrow S_2 \rightarrow S_3$$

A deadlock is detected at S_3 .

Recovery

Rollback from S_3 to S_1 :

$$S_3 \rightarrow S_1$$

Release resources and restart from S_1 .

- Work between S_1 and S_3 may need to be repeated.
- The resource allocation can be changed during re-execution.

Recovery Methods: Comparison

Method	Main action	Major concern
Process termination	Kill one or more processes	Loss of completed work
Resource preemption	Take resource from a process	Resource may not be preemptible
Rollback	Return to earlier checkpoint	Repeated computation / lost work

Break the deadlock while minimizing recovery cost

Detection and Recovery Trade-offs

Detection frequency

Frequent detection

- Faster discovery.
- More CPU/algorithm overhead.

Infrequent detection

- Lower overhead.
- Deadlock persists longer.

Recovery aggressiveness

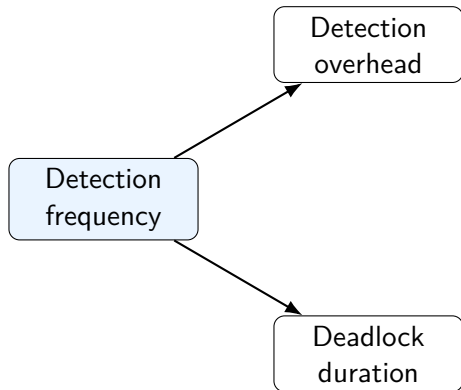
Aggressive recovery

- Faster resource release.
- Potentially larger work loss.

Conservative recovery

- May preserve more work.
- Recovery can take longer.

Detection Frequency: Design Trade-off



Balance

Increasing detection frequency generally increases detection overhead but can reduce the time a deadlock remains undetected. No single frequency is optimal for every workload.

Recovery Cost Factors

Recovery cost can depend on:

- Amount of completed computation that will be lost.
- Number of processes terminated or rolled back.
- Number and type of resources released.
- Cost of restarting processes.
- Cost of reconstructing consistent state.
- Impact on interactive users.
- Possibility of repeated victim selection.

System-level view

The cheapest action is not necessarily the one that terminates the fewest processes; it is the action that minimizes overall system cost.

Starvation During Recovery

Example

Suppose P1 is repeatedly selected as the recovery victim:

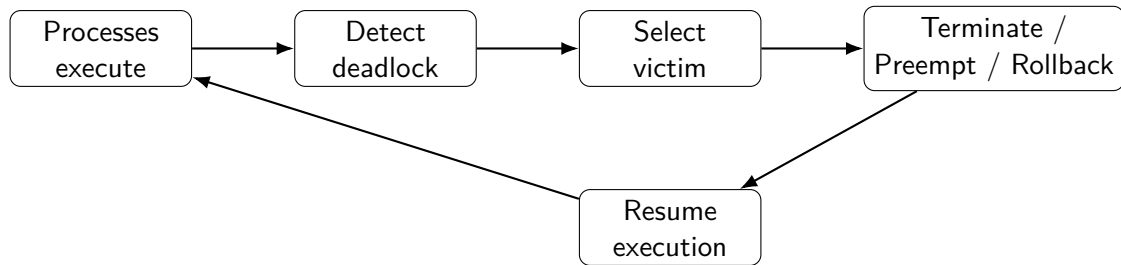
P1 preempted → restarts → deadlock → P1 preempted again

P1 may repeatedly lose its progress while other processes continue.

Mitigation

Include the number of previous rollbacks/preemptions as a factor in victim selection.

Detection + Recovery: Complete Flow



Correct detection + controlled recovery + starvation control

Exam-Oriented Questions

- ➊ Why is a cycle in a wait-for graph sufficient for deadlock with single-instance resources?
- ➋ Why is a cycle not sufficient with multiple resource instances?
- ➌ Compare process termination, resource preemption and rollback.
- ➍ What factors can be used to select a process for termination?
- ➎ Why is resource preemption difficult for some resources?
- ➏ Explain rollback using a checkpoint example.
- ➐ What is the trade-off between frequent and infrequent detection?
- ➑ How can recovery lead to starvation?

Summary

- **Wait-for graph**: process-only representation of waiting dependencies.
- **Single instance**: cycle detection identifies deadlock.
- **Multiple instances**: detection considers Available, Allocation and outstanding Requests.
- **Process termination**: releases resources but may lose work.
- **Resource preemption**: breaks dependencies but is not universally possible.
- **Rollback**: restores an earlier checkpoint but may require repeated computation.
- Detection and recovery balance overhead, response time, work loss, resource utilization and starvation.

Detect → Diagnose → Recover → Resume